

Lab 7

Analyse `api_app.py`

Start by analysing the 'api_app.py'.

Run the REST API server

Let's run the server. Press + -> Other -> Terminal and run

```
bash
uvicorn api_app:app --host 0.0.0.0 --port 8000
```

Let's interact with the REST server

```
import requests

r = requests.post(
    "http://localhost:8000/v1/orders",
    json={"item": "book", "quantity": 1}
)
print(r.json())

{'id': 1, 'item': 'book', 'quantity': 1}
```

The code above imports the requests library, which provides a simple interface for making HTTP requests in Python, then sends an HTTP POST request to `http://localhost:8000/v1/orders` with a JSON payload containing an item ("book") and a quantity (1). Argument `json={...}` automatically serialises the Python dictionary into JSON format and sets the appropriate Content-Type: application/json header. Variable `r` stores the server's response object, and `print(r.json())` parses the response body as JSON and prints it as a Python dictionary, assuming the server returns a valid JSON response.

Calling the first Local Language Model (LM)

First we will interact with the light LM, the llama 3.2 with 1 billion of parameters.

```
import requests
import os

OLLAMA_BASE = "http://ollama:11434"

def call_llm(prompt, model="llama3.2:1b"):
    payload = {
        "model": model,
        "prompt": prompt,
```

```

        "stream": False
    }
    r = requests.post(f"{OLLAMA_BASE}/api/generate", json=payload,
timeout=60)
    return r.json()["response"]

```

```
print(call_llm("Return JSON: {\n\"status\":\n\"ok\"}"))
```

Here's a simple "Hello, World!" JSON response:

```

```json
{"status": "ok"}
```

```

If you'd like a more elaborate response, here's an example of what I could return with some additional context or data:

```

```json
{
 "status": "ok",
 "message": "Hello, World!",
 "data": {
 "username": "johnDoe",
 "email": "johndoe@example.com"
 }
}
```

```

Let me know if you have any other requests.

Calling the second Local Language Model (LM)

Now let's interact with the second LM with 3 billion parameters.

```

import requests
import os

OLLAMA_BASE = "http://ollama:11434"

def call_llm(prompt, model="phi3:mini"):
    payload = {
        "model": model,
        "prompt": prompt,
        "stream": False
    }
    r = requests.post(f"{OLLAMA_BASE}/api/generate", json=payload,
timeout=60)
    return r.json()["response"]

print(call_llm("Return JSON: {\n\"status\":\n\"ok\"}"))

```

```
```json
{
 "status": "ok"
}
```
```

Analyse the Agentic Monitoring app

Open and analyse the `monitor.py`

Let's run the monitor with the 2x LMs

```
import subprocess
import time
import signal
import re
import json

def run_monitor(model_name, duration=30):
    cmd = ["python", "monitor.py", "--model", model_name]
    proc = subprocess.Popen(
        cmd,
        stdout=subprocess.PIPE,
        stderr=subprocess.STDOUT,
        text=True
    )

    try:
        time.sleep(duration)
        proc.send_signal(signal.SIGINT)
    except:
        proc.wait(timeout=5)
    except subprocess.TimeoutExpired:
        proc.kill()
    finally:
        output = proc.stdout.read() if proc.stdout else ""

    return output

def extract_assessments(log_text):
    pattern = r"Assessment:\s*(\{.*?\})"
    matches = re.findall(pattern, log_text)
    parsed = []
    for m in matches:
        try:
            parsed.append(json.loads(m.replace("'", '"')))
        except:
            pass
```

```

    return parsed

def summarize(assessments):
    summary = {"low": 0, "medium": 0, "high": 0, "critical": 0}
    for a in assessments:
        if "severity" in a:
            summary[a["severity"]] += 1
    return summary

duration = 30

print("Running llama3.2:1b...")
output_llama = run_monitor("llama3.2:1b", duration=duration)

print("Running phi3:mini...")
output_phi = run_monitor("phi3:mini", duration=duration)

assess_llama = extract_assessments(output_llama)
assess_phi = extract_assessments(output_phi)

print("\n=== llama3.2:1b assessments ===")
for a in assess_llama:
    print(a)

print("\n=== phi3:mini assessments ===")
for a in assess_phi:
    print(a)

print("\n=== Severity Summary ===")
print("llama3.2:1b:", summarize(assess_llama))
print("phi3:mini:", summarize(assess_phi))

Running llama3.2:1b...
Running phi3:mini...

=== llama3.2:1b assessments ===
{'severity': 'low', 'confidence': 0.5, 'action': 'none'}

=== phi3:mini assessments ===
{'severity': 'low', 'confidence': 1.0, 'action': 'none'}

=== Severity Summary ===
llama3.2:1b: {'low': 1, 'medium': 0, 'high': 0, 'critical': 0}
phi3:mini: {'low': 1, 'medium': 0, 'high': 0, 'critical': 0}

```

Let's now visualise stress scenarios

Analyse the `escalation_demo.py`

```
# Run ALL levels
!python escalation_demo.py --scenario all

# Run only Level 0 (log only)
!python escalation_demo.py --scenario level0

# Run only Level 1 (secondary model analysis)
!python escalation_demo.py --scenario level1 --light-model llama3.2:1b
--heavy-model phi3:mini

# Run only Level 2 (human notification)
!python escalation_demo.py --scenario level2
```

Level 0 – Log only

```
Context: {
  "timestamp": 1771797436.7636642,
  "error_rate": 0.0,
  "health": {
    "status": "ok",
    "time": 1771797436
  },
  "health_status": "ok",
  "slow": {
    "ok": true
  },
  "demo": {
    "goal": "show Level 0 behaviour"
  }
}
Action taken: none
```

Level 1 – Secondary model analysis ('heavy')

```
First-pass assessment: {'severity': 'low', 'confidence': 0.0,
'action': 'none'}
Policy decision: none
Secondary analysis not triggered (policy did not select 'heavy').
```

Level 2 – Human notification (policy override via deterministic signal)

```
Level 2 – Human notification failed via API; printing alert instead:
{
  "alert_id": "8efb84f39235ebe6460d8959",
  "source": "escalation_demo",
  "window_minute": 29529957,
  "assessment": {
    "severity": "high",
    "confidence": 0.9,
    "action": "escalate"
  },
  "context": {
```

```
    "timestamp": 1771797436.7636642,
    "error_rate": 0.25,
    "health": {
      "status": "ok",
      "time": 1771797436
    },
    "health_status": "ok",
    "slow": {
      "ok": true
    },
    "demo": {
      "goal": "force Level 2 via deterministic override"
    }
  }
}
```

Level 0 – Log only

```
Context: {
  "timestamp": 1771797444.179855,
  "error_rate": 0.0,
  "health": {
    "status": "ok",
    "time": 1771797444
  },
  "health_status": "ok",
  "slow": {
    "ok": true
  },
  "demo": {
    "goal": "show Level 0 behaviour"
  }
}
```

Action taken: none

Level 1 – Secondary model analysis ('heavy')

First-pass assessment: {'severity': 'low', 'confidence': 0.0, 'action': 'none'}

Policy decision: none

Secondary analysis not triggered (policy did not select 'heavy').

Level 2 – Human notification (policy override via deterministic signal)

Level 2 – Human notification failed via API; printing alert instead:

```
{
  "alert_id": "c20b34ac49ecb76a52b11ca4",
  "source": "escalation_demo",
  "window_minute": 29529957,
  "assessment": {
    "severity": "high",
    "confidence": 0.9,

```

```

    "action": "escalate"
  },
  "context": {
    "timestamp": 1771797452.5349755,
    "error_rate": 0.25,
    "health": {
      "status": "ok",
      "time": 1771797452
    },
    "health_status": "ok",
    "slow": {
      "ok": true
    },
    "demo": {
      "goal": "force Level 2 via deterministic override"
    }
  }
}

```

Let's analyse the results

Output shows three separate demonstrations of the escalation hierarchy, repeated twice because `--scenario` all already runs Level 0, Level 1, and Level 2 in sequence, and then subsequent commands (`--scenario level0`, `--scenario level1`, `--scenario level2`) repeat the same logic again.

Level 0 ("Log only")

printed the sensed context from the API and then took no action. Health returned `{"status":"ok"}`, slow endpoint returned `{"ok": true}`, and error rate was 0.0, so no alerting conditions existed. Result "Action taken: none" indicates pure observability with no escalation.

Level 1 ("Secondary model analysis")

did not trigger because the first-pass LLM output was `{'severity': 'low', 'confidence': 0.0, 'action': 'none'}`. Policy function `escalate_policy()` only selects the "heavy" second-pass path when severity is high or critical and confidence is below 0.7. Severity was low, so policy decision became none, and the code correctly skipped the second model call ("Secondary analysis not triggered"). Confidence being 0.0 is not used unless severity is high/critical, so it did not matter here.

Level 2 ("Human notification")

forced by a deterministic override in the demo: the context sets `error_rate` to 0.25, and policy says "human" whenever `error_rate > 0.2`. The script then attempted to send an idempotent PUT to `/v1/alerts/{alert_id}` with an `Idempotency-Key`. Message "failed via API; printing alert instead" indicates the PUT request did not succeed (common causes include the endpoint not existing, wrong route, method not allowed, server rejecting the header/body, or the service not running). Because of the failure, the script fell back to printing the alert payload locally, including the

computed alert_id, the minute window (window_minute), the forced assessment (high, 0.9, escalate), and the context snapshot used to justify the notification.

Alert IDs differ between the two Level 2 blocks because the payload includes timestamp, so each run produces a different hash even within the same window_minute. If identical IDs across runs are desired (true idempotency across repeats), the payload used for hashing should exclude volatile fields such as timestamp and include only stable fields (for example severity + status + minute window).

Let's now compare the output of both LMs results

```
import sys
import subprocess
import re
import ast
import pandas as pd
from collections import Counter
from IPython.display import display, clear_output

# -----
# Stress-test configuration
# -----
ITERATIONS = 10
TIMEOUT_SECONDS = 120

MODELS = ["llama3.2:1b", "phi3:mini"]

HEAVY_FIXED = "phi3:mini"
LIGHT_CONFIGS = [(m, HEAVY_FIXED) for m in MODELS]

LIGHT_FIXED = "llama3.2:1b"
HEAVY_CONFIGS = [(LIGHT_FIXED, m) for m in MODELS]

RUN_LIGHT_COMPARISON = True
RUN_HEAVY_COMPARISON = True

# -----
# Helpers
# -----
def run_level1(light_model: str, heavy_model: str) -> str:
    cmd = [
        sys.executable,
        "escalation_demo.py",
        "--scenario",
        "level1",
        "--light-model",
        light_model,
        "--heavy-model",
        heavy_model,
    ]
```



```

    p = subprocess.run(cmd, capture_output=True, text=True,
timeout=TIMEOUT_SECONDS)
    return (p.stdout or "") + ("\n" + p.stderr if p.stderr else "")

def parse_assessments(output: str):
    first = None
    second = None
    policy = None
    triggered = False

    m1 = re.search(r"First-pass assessment:\s*(\{.*\})", output)
    if m1:
        try:
            first = ast.literal_eval(m1.group(1))
        except Exception:
            first = None

    mpol = re.search(r"Policy decision:\s*([a-zA-Z]+)", output)
    if mpol:
        policy = mpol.group(1).strip()

    if "Secondary analysis not triggered" not in output:
        m2 = re.search(r"Second-pass assessment:\s*(\{.*\})", output)
        if m2:
            try:
                second = ast.literal_eval(m2.group(1))
                triggered = True
            except Exception:
                second = None
                triggered = True
        elif "Second-pass assessment:" in output:
            triggered = True

    return first, second, policy, triggered

def record_rows(label: str, light_model: str, heavy_model: str,
iteration: int, output: str):
    first, second, policy, triggered = parse_assessments(output)

    return {
        "experiment": label,
        "iteration": iteration,
        "light_model": light_model,
        "heavy_model": heavy_model,
        "policy_decision": policy,
        "secondary_triggered": triggered,
        "first_severity": None if not first else
first.get("severity"),
        "first_confidence": None if not first else
first.get("confidence"),

```

```

        "first_action": None if not first else first.get("action"),
        "second_severity": None if not second else
second.get("severity"),
        "second_confidence": None if not second else
second.get("confidence"),
        "second_action": None if not second else second.get("action"),
        "raw_output": output,
    }

def summarize(df: pd.DataFrame, title: str):
    print("\n" + "=" * 80)
    print(title)
    print("=" * 80)

    group_cols = ["experiment", "light_model", "heavy_model"]
    for (exp, lm, hm), g in df.groupby(group_cols):
        n = len(g)
        print(f"\nConfig: experiment={exp}, light={lm}, heavy={hm},
runs={n}")

        sev_counts = Counter([x for x in g["first_severity"].tolist()
if x is not None])
        act_counts = Counter([x for x in g["first_action"].tolist() if
x is not None])
        conf_vals = [c for c in g["first_confidence"].tolist() if
isinstance(c, (int, float))]

        print("First-pass severity counts:", dict(sev_counts))
        print("First-pass action counts:", dict(act_counts))
        if conf_vals:
            print(f"First-pass confidence
mean={sum(conf_vals)/len(conf_vals):.3f}")

        pol_counts = Counter([x for x in g["policy_decision"].tolist()
if x is not None])
        print("Policy decision counts:", dict(pol_counts))

        trig_rate = g["secondary_triggered"].mean() if n else 0.0
        print(f"Secondary triggered rate: {trig_rate:.2f}")

# -----
# Progress-aware execution
# -----
rows = []

total_runs = 0
if RUN_LIGHT_COMPARISON:
    total_runs += len(LIGHT_CONFIGS) * ITERATIONS
if RUN_HEAVY_COMPARISON:
    total_runs += len(HEAVY_CONFIGS) * ITERATIONS

```

```

current_run = 0

def print_progress():
    clear_output(wait=True)
    print(f"Progress: {current_run}/{total_runs} runs completed")
    print(f"Completion: {(current_run/total_runs)*100:.1f}%")

if RUN_LIGHT_COMPARISON:
    for light_model, heavy_model in LIGHT_CONFIGS:
        for i in range(1, ITERATIONS + 1):
            current_run += 1
            print_progress()
            print(f"Running (LIGHT comparison) | light={light_model}
heavy={heavy_model} | iteration={i}")
            out = run_level1(light_model, heavy_model)
            rows.append(record_rows("compare_light_models",
light_model, heavy_model, i, out))

if RUN_HEAVY_COMPARISON:
    for light_model, heavy_model in HEAVY_CONFIGS:
        for i in range(1, ITERATIONS + 1):
            current_run += 1
            print_progress()
            print(f"Running (HEAVY comparison) | light={light_model}
heavy={heavy_model} | iteration={i}")
            out = run_level1(light_model, heavy_model)
            rows.append(record_rows("compare_heavy_models",
light_model, heavy_model, i, out))

clear_output(wait=True)
print("Stress test completed.\n")

df = pd.DataFrame(rows)
display(df.drop(columns=["raw_output"]).head(20))

summarize(df[df["experiment"] == "compare_light_models"],
          "Stress test summary: comparing models as FIRST-PASS
(light)")

summarize(df[df["experiment"] == "compare_heavy_models"],
          "Stress test summary: comparing models as SECOND-PASS
(heavy)")

Stress test completed.

```

| | experiment | iteration | light_model | heavy_model |
|-------------------|----------------------|-----------|-------------|-------------|
| policy_decision \ | | | | |
| 0 | compare_light_models | 1 | llama3.2:1b | phi3:mini |
| heavy | | | | |

| | | | | |
|-------|----------------------|----|-------------|-----------|
| 1 | compare_light_models | 2 | llama3.2:1b | phi3:mini |
| none | | | | |
| 2 | compare_light_models | 3 | llama3.2:1b | phi3:mini |
| none | | | | |
| 3 | compare_light_models | 4 | llama3.2:1b | phi3:mini |
| heavy | | | | |
| 4 | compare_light_models | 5 | llama3.2:1b | phi3:mini |
| heavy | | | | |
| 5 | compare_light_models | 6 | llama3.2:1b | phi3:mini |
| heavy | | | | |
| 6 | compare_light_models | 7 | llama3.2:1b | phi3:mini |
| heavy | | | | |
| 7 | compare_light_models | 8 | llama3.2:1b | phi3:mini |
| none | | | | |
| 8 | compare_light_models | 9 | llama3.2:1b | phi3:mini |
| none | | | | |
| 9 | compare_light_models | 10 | llama3.2:1b | phi3:mini |
| heavy | | | | |
| 10 | compare_light_models | 1 | phi3:mini | phi3:mini |
| none | | | | |
| 11 | compare_light_models | 2 | phi3:mini | phi3:mini |
| none | | | | |
| 12 | compare_light_models | 3 | phi3:mini | phi3:mini |
| none | | | | |
| 13 | compare_light_models | 4 | phi3:mini | phi3:mini |
| none | | | | |
| 14 | compare_light_models | 5 | phi3:mini | phi3:mini |
| none | | | | |
| 15 | compare_light_models | 6 | phi3:mini | phi3:mini |
| none | | | | |
| 16 | compare_light_models | 7 | phi3:mini | phi3:mini |
| none | | | | |
| 17 | compare_light_models | 8 | phi3:mini | phi3:mini |
| NaN | | | | |
| 18 | compare_light_models | 9 | phi3:mini | phi3:mini |
| NaN | | | | |
| 19 | compare_light_models | 10 | phi3:mini | phi3:mini |
| none | | | | |

| | secondary_triggered | first_severity | first_confidence | first_action |
|---|---------------------|----------------|------------------|--------------|
| \ | | | | |
| 0 | True | high | 0.55 | escalate |
| 1 | False | low | 0.00 | none |
| 2 | False | low | 0.00 | none |
| 3 | False | high | 0.55 | escalate |
| 4 | False | high | 0.55 | escalate |

| | | | | |
|----|-----------------|-------------------|---------------|-------------|
| 5 | False | high | 0.55 | escalate |
| 6 | False | high | 0.55 | escalate |
| 7 | False | low | 0.00 | none |
| 8 | False | low | 0.00 | none |
| 9 | False | high | 0.55 | escalate |
| 10 | False | low | 0.00 | investigate |
| 11 | False | low | 1.00 | none |
| 12 | False | low | 0.50 | investigate |
| 13 | False | low | 0.00 | investigate |
| 14 | False | low | 0.50 | investigate |
| 15 | False | critical | 0.70 | investigate |
| 16 | False | critical | 0.90 | escalate |
| 17 | False | NaN | NaN | NaN |
| 18 | False | NaN | NaN | NaN |
| 19 | False | critical | 0.90 | investigate |
| | second_severity | second_confidence | second_action | |
| 0 | NaN | NaN | NaN | |
| 1 | NaN | NaN | NaN | |
| 2 | NaN | NaN | NaN | |
| 3 | NaN | NaN | NaN | |
| 4 | NaN | NaN | NaN | |
| 5 | NaN | NaN | NaN | |
| 6 | NaN | NaN | NaN | |
| 7 | NaN | NaN | NaN | |
| 8 | NaN | NaN | NaN | |
| 9 | NaN | NaN | NaN | |
| 10 | NaN | NaN | NaN | |
| 11 | NaN | NaN | NaN | |
| 12 | NaN | NaN | NaN | |
| 13 | NaN | NaN | NaN | |
| 14 | NaN | NaN | NaN | |
| 15 | NaN | NaN | NaN | |
| 16 | NaN | NaN | NaN | |
| 17 | NaN | NaN | NaN | |

| | | | |
|----|-----|-----|-----|
| 18 | NaN | NaN | NaN |
| 19 | NaN | NaN | NaN |

=====

=====

Stress test summary: comparing models as FIRST-PASS (light)

=====

=====

Config: experiment=compare_light_models, light=llama3.2:1b,
heavy=phi3:mini, runs=10

First-pass severity counts: {'high': 6, 'low': 4}

First-pass action counts: {'escalate': 6, 'none': 4}

First-pass confidence mean=0.330

Policy decision counts: {'heavy': 6, 'none': 4}

Secondary triggered rate: 0.10

Config: experiment=compare_light_models, light=phi3:mini,
heavy=phi3:mini, runs=10

First-pass severity counts: {'low': 5, 'critical': 3, nan: 2}

First-pass action counts: {'investigate': 6, 'none': 1, 'escalate': 1,
nan: 2}

First-pass confidence mean=nan

Policy decision counts: {'none': 8, nan: 2}

Secondary triggered rate: 0.00

=====

=====

Stress test summary: comparing models as SECOND-PASS (heavy)

=====

=====

Config: experiment=compare_heavy_models, light=llama3.2:1b,
heavy=llama3.2:1b, runs=10

First-pass severity counts: {'low': 9, 'high': 1}

First-pass action counts: {'none': 7, 'investigate': 2, 'escalate': 1}

First-pass confidence mean=0.055

Policy decision counts: {'none': 9, 'heavy': 1}

Secondary triggered rate: 0.10

Config: experiment=compare_heavy_models, light=llama3.2:1b,
heavy=phi3:mini, runs=10

First-pass severity counts: {'low': 5, 'high': 5}

First-pass action counts: {'none': 4, 'escalate': 5, 'investigate': 1}

First-pass confidence mean=0.275

Policy decision counts: {'none': 5, 'heavy': 5}

Secondary triggered rate: 0.00

Let's analyse the results

The comparison of models as first-pass ("light") triage components reveals a marked behavioural difference between llama3.2:1b and phi3:mini. When llama3.2:1b is used as the light model, it produces six "high" and four "low" severities across ten runs, triggering the heavy policy decision in 60% of cases. Its mean confidence is relatively low (0.33), which explains the frequent escalation under a confidence-sensitive policy. By contrast, when phi3:mini is used as the light model, it produces predominantly "low" severities (five low, three critical, two NaN), and the policy decision is "none" in eight out of ten runs. No secondary stage is triggered in this configuration. However, the presence of NaN severities and NaN confidence values suggests occasional malformed or inconsistent outputs, which is operationally significant. The absence of secondary triggering combined with occasional critical severities indicates a more conservative escalation behaviour but potentially weaker structural reliability.

When comparing heavy-model configurations, different patterns emerge. Using llama3.2:1b as both light and heavy produces predominantly "low" severities (nine low, one high) with a very low mean confidence (0.055). Only one case escalates to heavy processing, and the secondary trigger rate remains modest (0.10). In contrast, pairing llama3.2:1b (light) with phi3:mini (heavy) yields a balanced distribution (five low, five high) and a higher mean confidence (0.275), with heavy escalation occurring in 50% of cases. Notably, the secondary triggered rate is zero, implying that policy logic rather than model instability governs escalation in this configuration. Overall, llama3.2:1b appears more escalation-prone as a first-pass model due to lower confidence outputs, whereas phi3:mini exhibits greater stability in policy decisions but occasional structural inconsistencies that must be addressed through stricter schema validation.

Task: Build and Consume a Simple REST API

Design and implement a REST API server `task_rest_server.py` that:

- Accepts three inputs: A, B, and C
- Computes the equation $result = (A+B) \times C$
- Returns the computed result in JSON format
- Returns appropriate HTTP error codes if A, B, or C are not valid integers or floating-point numbers

The API must then be accessed from a cell.

Part 1: Requirements for the REST API Server

Implement a POST endpoint `/v1/calculate`

The request body must be JSON:

```
{
  "A": 1,
  "B": 2,
  "C": 3
}
```

The response for valid input must be:

```
{
  "A": 1,
  "B": 2,
  "C": 3,
  "result": 9
}
```

If any of A, B, or C:

- Are missing
- Are not integers or floating-point numbers
- Cannot be converted to numeric type

The API must return:

- HTTP 400 (Bad Request)
- JSON error message explaining the issue

Part 2: Implementation Constraints

- Use FastAPI.
- Use proper validation.
- Use appropriate HTTP status codes.
- Ensure the API runs on port 8000.

Part 3: Example Expected Server Behaviour

Valid request example

Input:

```
{
  "A": 2,
  "B": 3,
  "C": 4
}
```

Output:

```
{
  "A": 2,
  "B": 3,
  "C": 4,
  "result": 20
}
```


Invalid request example

Input:

```
{
  "A": "two",
  "B": 3,
  "C": 4
}
```

Output:

```
{
  "detail": "A, B and C must be numeric values"
}
```

Status code:

400

Part 4: Jupyter Notebook Client Code

Before running the server, ensure that no other server is running. To run the server

```
uvicorn task_rest_server:app --host 0.0.0.0 --port 8000
```

Use the code below to test the application

```
import requests
from pprint import pprint

API_BASE = "http://localhost:8000"

def post_calculate(A, B, C):
    payload = {"A": A, "B": B, "C": C}
    r = requests.post(f"{API_BASE}/v1/calculate", json=payload,
                      timeout=10)
    return r

# 1) Valid request
resp = post_calculate(2, 3, 4)
print("Valid request status:", resp.status_code)
pprint(resp.json())

# 2) Another valid request with floats
resp = post_calculate(1.5, 2.0, 10)
print("\nFloat request status:", resp.status_code)
pprint(resp.json())
```

```

# 3) Invalid request: non-numeric A
resp = post_calculate("two", 3, 4)
print("\nInvalid request status:", resp.status_code)
pprint(resp.json())

# 4) Invalid request: missing field example (send custom payload)
bad_payload = {"A": 1, "B": 2} # missing C
resp = requests.post(f"{API_BASE}/v1/calculate", json=bad_payload,
timeout=10)
print("\nMissing field status:", resp.status_code)
pprint(resp.json())

Valid request status: 200
{'A': 2.0, 'B': 3.0, 'C': 4.0, 'result': 20.0}

Float request status: 200
{'A': 1.5, 'B': 2.0, 'C': 10.0, 'result': 35.0}

Invalid request status: 422
{'detail': [{'input': 'two',
                  'loc': ['body', 'A'],
                  'msg': 'Input should be a valid number, unable to parse
string as '
                  'a number',
                  'type': 'float_parsing'}]}}

Missing field status: 422
{'detail': [{'input': {'A': 1, 'B': 2},
                  'loc': ['body', 'C'],
                  'msg': 'Field required',
                  'type': 'missing'}]}}

```